

SECTION – AThere are **FOUR** questions in this section. Answer any **THREE**.

1. (a) Derive the time complexity of the following code fragments:

(4×4=16)

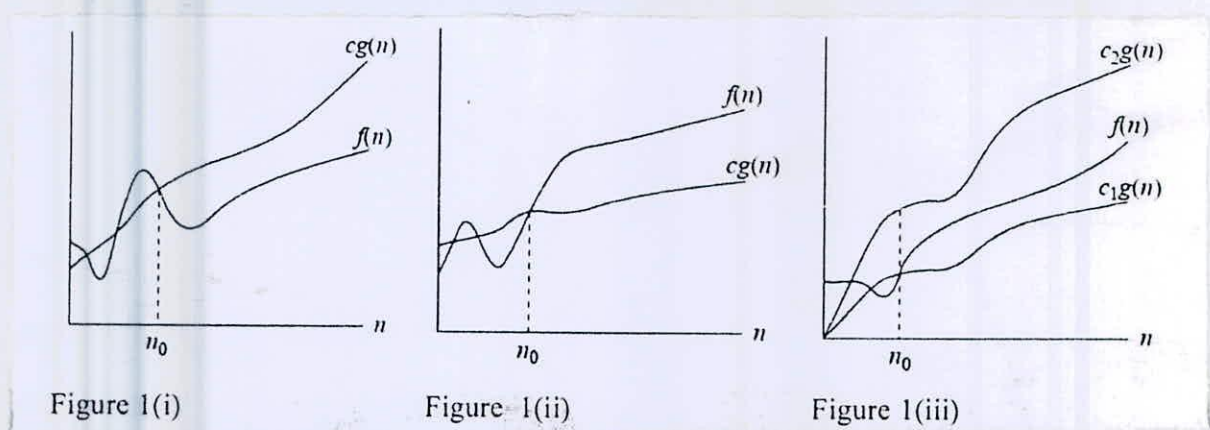
Code Fragment 1 <pre>for (i = 0; i < n; ++i) { for (j = 0; j < i; ++j) { sum += i + j; } }</pre>	Code Fragment 2 <pre>for (i = 0; i < n; ++i) { for (j = 0; j < i*i; ++j) { sum += i + j; } }</pre>
Code Fragment 3 <pre>for (i = 0; i < n*n; ++i) { for (t j = 0; j < i; ++j) { sum += i + j; } }</pre>	Code Fragment 4 <pre>sum = 0; for (k=1; k<=n; k*=2) { for (j=1; j<=k; j++) { sum++; } }</pre>

- (b) In an array based implementation of a stack when the array space runs out, the array capacity is increased. Now, analyze and explain the following scenarios through an amortized analysis of the push() function.

(2×7=14)

- (i) Increasing the Capacity of the Array by 1
(ii) Doubling the Capacity of the Array

(c)

(5)

Explain what do the three figures, i.e., Figure 1(i–iii) signify with respect to asymptotic notations.

CSE 105/CSE

2. (a) You are given the following numbers to insert into an empty Binary Search Tree (BST):

(5)

5, 7, 8, 12, 15, 27

Determine which insertion order below would yield the tree with the least height?

- (i) 15, 5, 27, 8, 7, 12
- (ii) 12, 7, 15, 27, 5, 8
- (iii) 8, 27, 7, 5, 15, 12
- (iv) 7, 5, 12, 8, 15, 27

(b) A perfect balanced BST requires all interior nodes to contain two children and all leaf nodes to be on the same level. A perfect balanced BST of $h = 0$ would have $n = 1$ node, $h = 1$ would have $n = 3$ nodes, $h = 2$ would have $n = 7$ nodes, and so on. Here, h is tree height and n is the number of nodes in the tree. Providing appropriate arguments, derive the worst-case running time, in terms of n , for successfully searching for an element in a perfect balanced BST.

(10)

(c) Present a recursive method *countInRange(root, low, high)*, without any helper functions, to count, in a BST, all keys that are within a given range. Assume that keys in the BST are unique, i.e., there are no duplicates. For example, suppose that the keys in a BST are 4, 7, 10, 14, 15, 17, 20, 30. Then, *countInRange(root, 8, 17)* returns 4 and *countInRange(root, 21, 29)* returns 0. For your convenience a code snippet in C++ is given below, but, you need not follow any particular programming language.

(10)

```
public class BSTNode {
    int key;
    Value value;
    BSTNode left;
    BSTNode right;
    ...
}
// counts number of keys in BST that are in the range low to high, inclusive
// i.e. all keys k such that low <= k <= high
public static int countInRange(BSTNode root, int low, int high) {
    // COMPLETE THIS METHOD
}
```

(d) Present an algorithm to delete a full node in a binary search tree. Explain how it works with illustrative examples.

(10)

3. (a) Suppose the numbers 0, 1, 2, ..., 9 were pushed onto a stack in that order, but that pops occurred at random points between the various pushes. Now you randomly wrote the following two sequences, S1 and S2 as example sequences in which the values in the stack could have been popped:

(5)

S1: 3, 2, 6, 5, 7, 4, 1, 0, 9, 8

S2: 3, 2, 6, 4, 7, 5, 1, 0, 9, 8

Noticing this, your genius little brother winked at you and said that only one of your sequences is valid! Can you determine why?

CSE 105/CSE

Contd.... for Q. No. 3

(b) Consider the following C++ fragment. A stack is being used to parse the matching parentheses / brackets / braces- (, [, and {. Show the state of the stack at the end of this fragment.

(5)

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     const int M = 30;
6     int N = 1024;
7     int parent[N], height[N], depth[N];
8     int maxheights[100];
9
10    for ( int i = 0; i < 100; ++i ) {
11        maxheights[i] = 0;
12    }
13
14    double minmeandepth = 1e300;
15    double meandepths = 0;
16    double maxmeandepth = 0;
17
18    for ( int j = 0; j < M; ++j ) {
19        for ( int i = 0; i < N; ++i ) {
20            parent[i] = -1;
21            height[i] = 0;
22        }
23
24        for ( int i = 0; i < N - 1; ++i ) {
25            while ( true ) {
26                int p1 = rand() & (N - 1);
27                int p2 = rand() & (N - 1);
28
29                int s1 = p1;
30                int s2 = p2;
31
32                while ( parent[s1

```

(c) You are given access to the following functions of a list data structure:

(25)

- *init(n)* /* Initialize an array $L[0..n - 1]$ of length n for integers. Current position 'points to' $L[0]$. All the functions below are applied on this array which is not accessible directly. */
- *insert(item)* /* Inserts an element at the current position and current position is 'incremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k + 1]$). Here, *item* is the element to be inserted. In case the list is full, it returns ERR (a large negative constant). */
- *remove()* /* Removes the element at the current position and current position is 'decremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k - 1]$). In case the list is empty, it returns ERR (a large negative constant). */
- *value()* /*Returns the value of the element at the current position. Current position remains unchanged. In case the list is empty, it returns ERR (a large negative constant). */

CSE 105/CSE

Contd.... for Q. No. 3(c)

- *moveToStart()* /*Sets the current position to $L[0]$.*/
- *moveToEnd()* /*Sets the current position to $L[n - 1]$.*/
- *next()* /*Increments the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k + 1]$). If $k = n - 1$, it returns ERR (a large negative constant).*/
- *prev()* /*Decrements the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k - 1]$). If $k = n - 1$, it returns ERR (a large negative constant).*/

Now you have to build one queue and one stack in one List data structure as defined above. The queue should grow from the beginning of the list (i.e., first element of the queue should be at $L[0]$) and the stack should grow from the end of the list (i.e., the first element of the stack should be at $L[n - 1]$). You have no access to the code of the above implementation and can only call these functions in your code. You need to write appropriate codes (no specific programming language is required; pseudo code is fine) for implementing the queue and stack simultaneously as described above. Please explain your code by giving appropriate comments. In particular, you need to implement the following four functions:

- *push(item)* /* push a positive integer, *item* in the stack. If there is an error (e.g., no more space available for growing the stack in the list, it should output an appropriate error message and returns with -1.*/
- *pop()* /* pop an element from the stack. If there is an error, it should output an appropriate error message and returns with -1.*/
- *enqueue(item)* /* enqueue a positive integer, *item* in the queue. If there is an error (e.g., no more space available for growing the queue in the list, it should output an appropriate error message and returns with -1.*/
- *dequeue()* /* dequeue an element from the queue. If there is an error, it should output an appropriate error message and returns with -1.*/

4. (a) Compute the best alignments (i.e., alignments with the minimum edit distance) of two DNA sequences, ATCGGT and ACCGT using dynamic programming. Use 2 as the gap penalty, 1 as the mismatch penalty, and 0 as the penalty for a match. If there are multiple optimal alignments, find all of them. You do not need to write the algorithm, just show the dynamic programming (DP) table. Show the alignments and mark the paths (which correspond to the alignments), in the DP table. (18)

(b) Given two strings, the longest common subsequence problem is to find the longest subsequence present in both strings. A common subsequence is a sequence that appears in both string in the same relative order, but not necessarily contiguous. For example, for two strings $S1 = \text{"ALGORITHM"}$ and $S2 = \text{"LITHIUM"}$, the longest common subsequence is "LITHM". Present a dynamic programming algorithm to find the length of the longest common subsequence of two strings. Derive a proof of correctness of the algorithm and also derive its running time. (8+6+3=17)

SECTION – B

There are **FOUR** questions in this section. Answer any **THREE** questions.

5. (a) Consider the following array (A) of integers. (20)

	1	2	3	4	5	6	7	8	9	10
A:	16	4	10	7	14	9	3	2	8	1

Verify whether the array is a max heap. Derive the complexity of your verification algorithm. Justify whether the algorithm $\text{max_heapify}(A, i)$ will establish the max heap property of the array, if you apply algorithm with index $i = 2$. Analyze the time complexity of max_heapify algorithm.

(b) Explain why topological sort algorithm cannot run on a directed graph that cycle(s). Derive the component graph of the directed graph G represented by the adjacency matrix give in **Fig. for Q. No. 5(b) and 6**. Then find the topological sorting of the vertices of the component graph. Show every steps. (15)

6. (a) Consider the adjacency matrix representation of a directed graph G given in **Fig. for Q. No. 5(b) and 6**. Draw the graph and find its adjacency list representation considering the vertices in ascending order. Find the shortest path distance of every vertex from vertex 1 using BFS. Show the queue status when the vertex 10 is just enqueued. Prove that the distance of the most recent vertex (tail in the queue) is at most one larger than the distance of the oldest vertex (head in the queue). (20)

(b) Classify the edges of the graph G represented by the adjacency matrix given in **Fig. for Q. No. 5(b) and 6** after you run DFS on the graph considering the vertices in ascending order. Prove that an edge (u, v) is a tree edge. (15)

$$\text{if } u.d < v.d < v.f < u.f.$$

Notations bear their usual meaning.

7. (a) Given a set $U = \{x_1, x_2, \dots, x_n\}$ of n integers and an integer k ($k < n$), give an optimal greedy algorithm to find a subset S of size k in U such that the summation of the numbers in S is maximum (over all the subsets of size k in U). Prove the correctness of your algorithm, and analyze the running time. (15)

(b) Given a sequence of n numbers $a_1, \dots, a_n$, which we assume are all distinct, we define an inversion to be a pair $i < j$ such that $a_i > a_j$. Present an $O(n \log n)$ divide-and-conquer algorithm to count the number of inversions between two orderings. One might feel that this measure of inversion is too sensitive and may want to change it. Let's call a pair a modified-inversion if $i < j$ and $a_i > 2a_j$. What modifications do you need to make in the divide-and-conquer algorithm (for counting inversion) to count the number of modified-inversions? (20)

CSE 105/CSE

8.

(a) Given n points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ in the plane, give an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm.

(b) Calculate the number of swap operations (data interchanges) required if you apply Build_Max_Heap algorithm on the array $A = \{5, 3, 17, 10, 84, 19, 6, 22, 9\}$. Justify whether the max heap you produced can be used as a priority queue. Show that Heap_Extract_Max algorithm is essentially a part of Heapsort algorithm. Derive and compare the time complexities of these two algorithms. (Heapsort and Heap_Extract_Max).
- (18)

(17)

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Fig. for Q. No. 5(b) and 6

SECTION – A

There are **FOUR** questions in this section. Answer any **THREE**.

1. (a) Discuss the difference between a class and an object. Describe how C++ supports inheritance. (11)
- (b) Demonstrate the order of execution of constructor functions for multilevel inheritance with an example program. (11)
- (c) Differentiate between 'new' operator and 'malloc()' function? Design "Class A" so that the following code inside the main function does not produce any error: (13)

```
int main()
{
    A obj(10);
    A *array = new A[10];
    return 0;
}
```

2. (a) Describe the differences between function overloading and function overriding. (10)
- (b) Design your custom inserter in the program below to generate the following output:
x: 120, y: 130 (10)

```
#include <iostream>
using namespace std;
class myclass{
    int x, y;
public:
    myclass(int x, int y){
        this->x=x;
        this->y=y;
    }
};
```

```
int main(){
    myclass ob(120, 130);
    cout<<ob;
    return 0;
}
```

CSE 107

Contd.... for Q. No. 2

(c) Consider the following class:

(15)

```
class strtype {
    char *p;
    int len;
public:
    strtype(char *s) {
        len = strlen(s) + 1;
        p = new char[len];
        strcpy(p, s);
    }
    ~strtype() {
        delete [ ] p;
    }
};
```

Identify the problem that may arise if we want to execute the following instructions from the main function:

```
strtype s1("LEVEL"), s2("TERM");
s1 = s2;
```

Without changing the constructor and the destructor function, write the code to solve the problem.

3. (a) Describe the use of a virtual base class. Explain why it is not possible to create an object of an abstract class? (12)

(b) Write an example program to demonstrate the use of forward declaration of a class. (11)

(c) Consider the following overloaded functions and develop the equivalent function using default arguments: (6)

```
void f2( ) { int a = 0, b = 0; ... }
void f2( int a ) { int b = 0; ... }
void f2( int a, int b ) { ... }
```

(d) Evaluate the output of the following program: (6)

<pre>#include <iostream> using namespace std; class circle{ int r; public: circle(int a=0) { r=a; } void show(){ cout << r << endl; } friend circle operator++(circle &ob); };</pre>	<pre>circle operator++(circle &ob) { ob.r++; return ob; } int main() { circle c1(10); ++c1; c1.show(); return 0; }</pre>
--	--